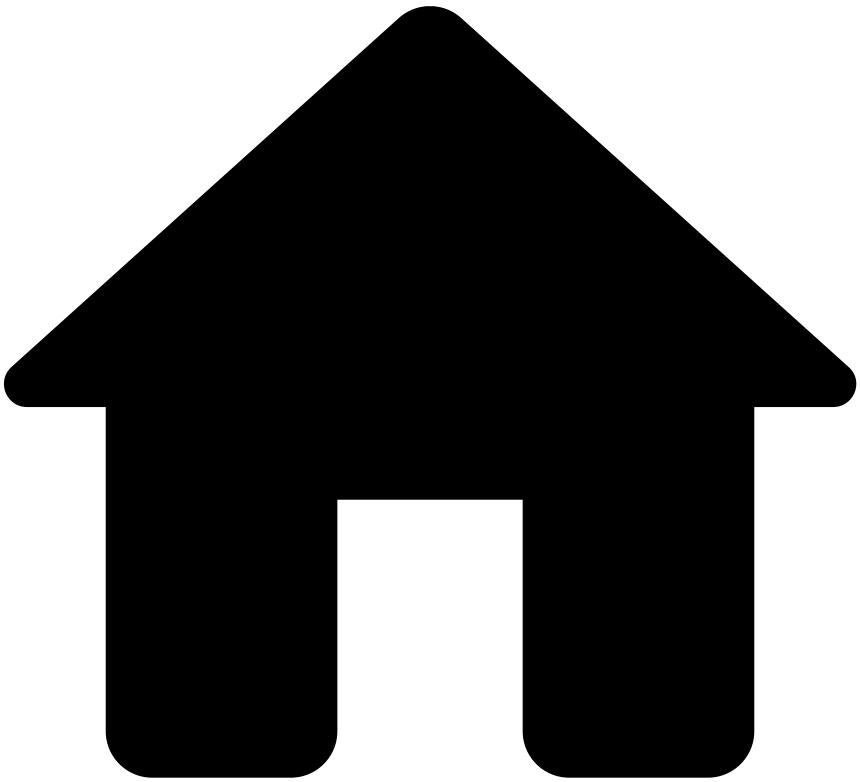


Reference Data

You can use the Reference Data API to send key business information to Feedzai, such as customer and account data. The reference data is then stored in the database, i.e. Reference Data Storage, and can be used to enrich events in real time, i.e. populate information from the reference data onto the transaction. This allows you to reduce the additional information that you would otherwise send along with each event.

As a best practice, you should send all entity data via Reference Data API. The reference data operations section explains which operations you should execute for data storage and which entities you should send to the Reference Data Storage for further event enrichment.

As a result, other Feedzai APIs can fetch this reference data from the Reference Data Storage each time the Pulse runtime workflow processes an event. Take into consideration that this is only possible if the sent keys have a match in the reference data storage.

Data storageâ□□

The following image shows the path of the reference data from the customer to Feedzai's Reference Data Storage:

The information sent via Reference Data API is organized into different entities (e.g. Account entity, Customer entity). Continue reading to learn how to store or update data in the Reference Data Storage to further enrich real-time events.

Reference data operationsâ□□

The following operations can be used with Reference Data API to get, store, update (in bulk or selectively), and delete data:

Method	HTTP request (example)	Purpose
`GET`	`GET <>/api/referenceData/payees/{key}`	Populates entity-related information in Case Manager.
`PUT`	`PUT <>/api/referenceData/customers/{key}`	Updates the full list of fields from an entity in the Reference Data Storage. This means that all entity fields are stored or updated in the Reference Data Storage. Ensure you send all entity-related fields when providing updates about an entity to avoid potential issues with the `PUT` method treating missing fields as removals.
`PATCH`	`PATCH <>/api/referenceData/cards/{key}`	Updates a field, or selection of fields from an entity in the Reference Data Storage. In contrast to `PUT`, which updates an entity in the Reference Data Storage in bulk, `PATCH` allows you to update only the fields you need.
`DELETE`	`DELETE <>/api/referenceData/credentials/{key}`	Deletes an entity from the Reference Data Storage.

To store, update or delete data from the Reference Data Storage, refer to the following table:

Endpoint	Key	Purpose
New Payees	`customer_id`	Enriches the Payee's entity data in the Reference Data Storage. As such, all updates to a payee must be done through Reference Data's Payees endpoint, through: - **`PUT <> /api/referenceData/payees/{key}`**, for an integral field update - **`PATCH <> /api/referenceData/new_payee/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <> /api/referenceData/payees/{key}`**.
Credentials	`customer_id`	Enriches the Credential's entity data in the Reference Data Storage. As such, all updates to those credentials must be done through Reference Data's Credentials endpoint, through: - **`PUT <> /api/referenceData/credentials/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/credentials/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/credentials/{key}`**.
Cards	`card_id`	The Cards endpoint is necessary both to populate the fields in Case Manager's **Customer** page and to enrich the Card's entity data in the Reference Data Storage. As such, all updates to a card must be done through Reference Data's

Endpoint	Key	Purpose
		Cards endpoint, through: - **`PUT <> /api/referenceData/cards/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/cards/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/cards/{key}`**.
Devices	`device_id`	Enriches the Device's entity data in the Reference Data Storage. As such, all updates to a device must be done through Reference Data's Devices endpoint, through: - **`PUT <> /api/referenceData/devices/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/devices/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/devices/{key}`**.
Customers	`sender_id` and `receiver_id`	The Customers endpoint is necessary both to populate the fields in Case Manager's **Customer** page and to enrich the Customer's entity data in the Reference Data Storage. As such, all updates to a customer must be done through Reference Data's Customers endpoint, through: - **`PUT <> /api/referenceData/customers/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/customers/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/customers/{key}`**.
Aliases	`customer_id`	Enriches the Alias's entity data in the Reference Data Storage. As such, all updates to an alias must be done through Reference Data's Aliases endpoint, through: - **`PUT <> /api/referenceData/aliases/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/aliases/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/aliases/{key}`**.
Merchants	`acceptor_id`	Enriches the Merchant's entity data in the Reference Data Storage. As such, all updates to a merchant must be done through Reference Data's Merchants endpoint, through: - **`PUT <> /api/referenceData/merchants/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/merchants/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/merchants/{key}`**.
Accounts	`account_iban` for cards `sender_account_iban` **and** `receiver_account_iban` for inbound and outbound transfers	Enriches the Account's entity data in the Reference Data Storage. As such, all updates to an account must be done through Reference Data's Accounts endpoint, through: - **`PUT <> /api/referenceData/accounts/{key}`**, for an integral field update. - **`Patch <> /api/referenceData/accounts/{key}`**, for updates to a selection of fields. To remove this entity from the Reference Data Storage you must delete it through: - **`DELETE <>/api/referenceData/accounts/{key}`**.

Real-life scenarioâ□□

Imagine that Feedzai receives a card authorization request, where the final output (i.e. payload) includes new information about `card_category`. This information will be present at the root of the payload.

However, the customer array (i.e. `customer_cards`) will not reflect the same changes. That is because this array is part of the customer information that is sent by the client to Feedzai through Reference Data API's Customer entity. To update arrays such as `customer_cards` or `customer_accounts`, you must update the Reference Data API's Customer entity through either `PUT` or `PATCH`. Refer to the Reference Data Operations section to understand the difference between these methods.

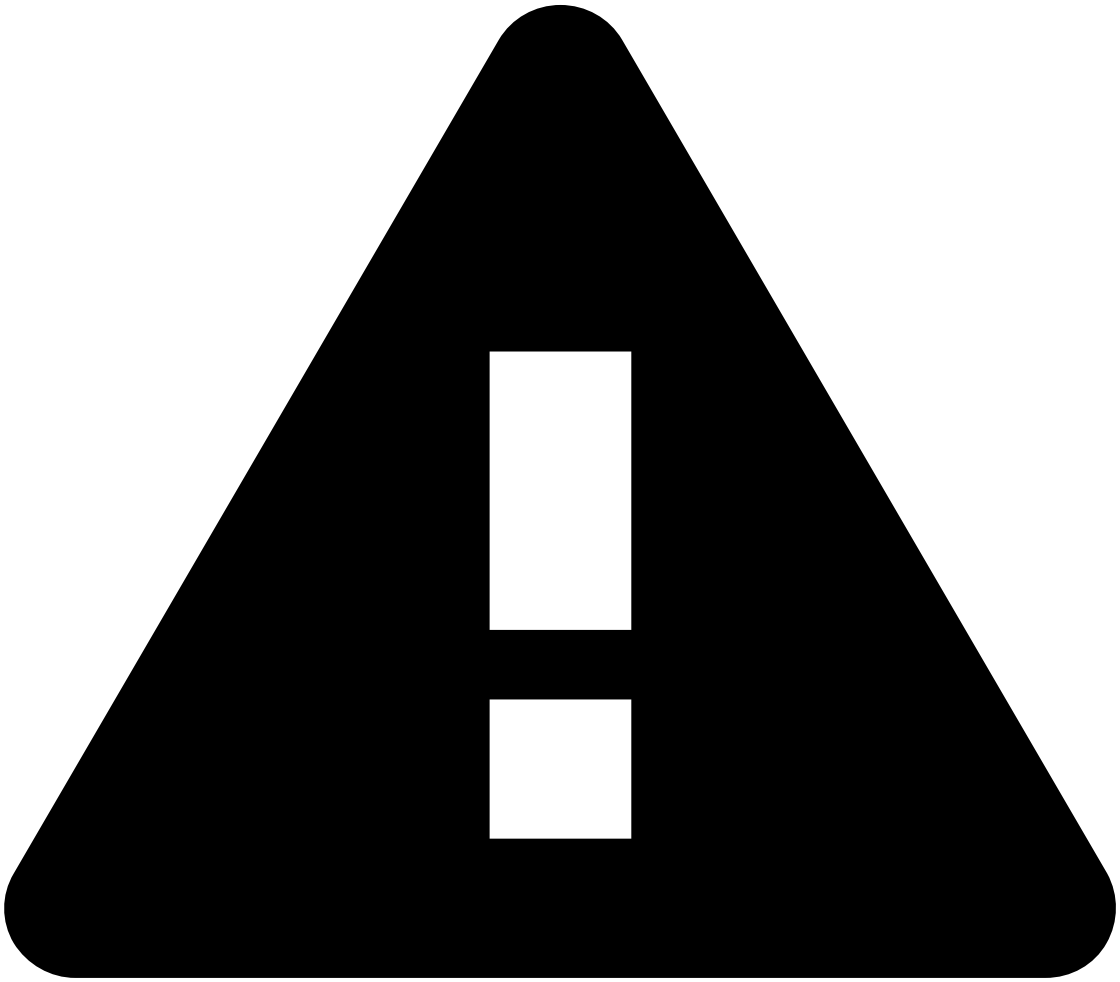
Digital Activity's use caseâ□□

You can trigger updates to the Reference Data Storage via the Digital Activity API. These updates apply exclusively to fields starting with the `new_*` prefix as described in the following examples:

- The customer wants to update the status of his card. He opens the app and momentarily disables the card. The bank calls the Update Card endpoint in Digital Activity and sets `new_card_status` = `blocked`. If approved, this will update the `card_status` for the Card entity in the Reference Data Storage.
- The customer changes his phone number on the bank's app. The bank sends a Profile Update request to Feedzai with `new_customer_phone_number` = `1-970-664-2654`. If approved, this will update the `customer_phone_number` = `1-970-664-2654` for the Customer entity in the Reference Data Storage.

Notice that updates from the Digital Activity API occur under the following conditions:

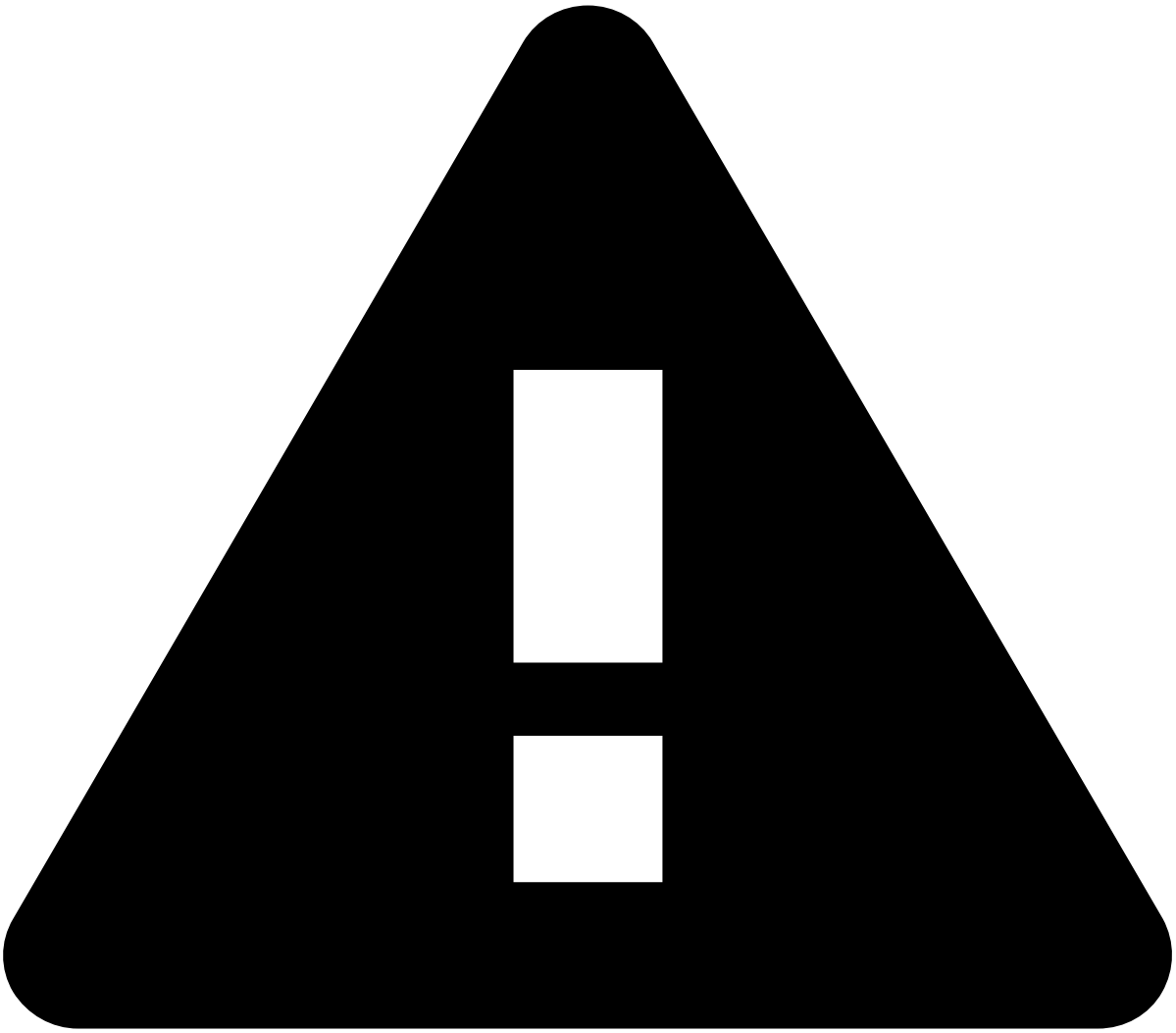
- `POST` events whose system decision is `approve`.
- `PUT` events regardless of the system decision.



caution

To amend information, you need to resend the entire entity data (i.e. populate all the `new_*` fields). If you want to update pieces of information instead of updating the entire entity, use the Reference Data API with the PATCH method. Refer to the Reference Data Operations section to understand the difference between these methods.

Real-time event enrichmentâ□□



caution

Real-time event enrichment can only occur if the relevant data (e.g. card, account, device) already exists in the Reference Data Storage. Learn how to store data in the Reference Data Storage using Reference Data's API endpoints.

When processing an event, if the Reference Data Storage contains more data about the entity whose event relates to, this extra data enriches the event. Take into consideration that this is only possible if the keys sent in the event (e.g. card_id, customer_id) have a match in the reference data storage.

The following image shows the field enrichment process for a payment initiation request.

The following scenarios might occur during the enrichment:

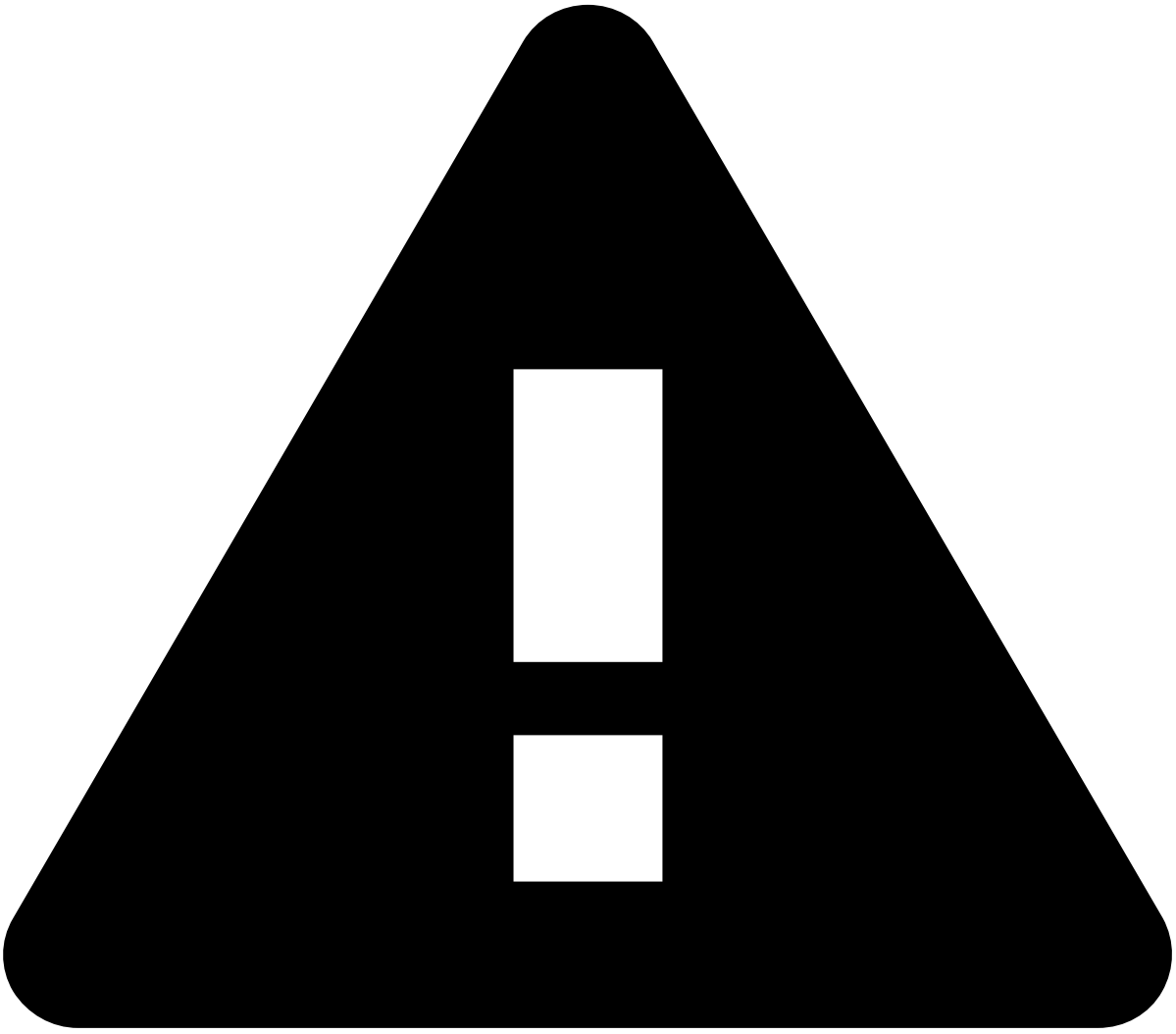
- **Scenario 1:** The field sent via the Transaction API (i.e. original field) is not populated, and the information is not in the Reference Data Storage. As a result, the field remains unpopulated.

- **Scenario 2:** The original field is not populated, but the corresponding field in the Reference Data Storage contains data. As a result, the field from the event is enriched with reference data. This is important for scoring purposes, as well as to help analysts throughout their investigation (i.e. the additional data populates Case Manager's fields).
- **Scenario 3:** The original field is populated with a different value than the value that is in the Reference Data Storage. As a result, the value sent via **the Transaction API prevails over the reference data**. To update the Reference Data Storage with the value sent via the Transaction API, you must use the Reference Data API to update the corresponding entity, namely through the `PATCH` method (see the difference between the `PATCH` and `PUT` methods).

Entitiesâ□□

To enrich the event with reference data as described in Scenario 2, the ID sent in the event must be associated with the entity that you want to enrich. For instance, to enrich the real-time event with card data, its `card_id` must match the `card_id` of Reference Data Storage.

The following sections show which entities can be used to enrich each event type with reference data.



caution

Note that for real-time event enrichment to take place, the relevant data must already exist in the Reference Data Storage. Learn how to store data in the Reference Data Storage using Reference Data's API endpoints.

Digital activity eventsâ

- **Payee data**: `new_payees`
  - **Key**: `customer_id`
- **Credential data**: `credentials`
  - **Key**: `customer_id`
- **Device data**: `devices`
  - **Key**: `device_id`
- **Alias data**: `aliases`
  - **Key**: `customer_id`
- **Profile data**: `customers`
  - **Key**: `customer_id`
- **Card data**: `cards`
  - **Key**: `card_id`
- **Account data**: `accounts`
  - **Key**: `account_iban`

Card eventsâ

- **Card data**: `cards`
  - **Key**: `card_id`
- **Customer data**: `customers`
  - **Key**: `customer_id`
- **Merchant data**: `merchants`
  - **Key**: `acceptor_id`
- **Account data**: `accounts`
  - **Key**: `account_iban`

Inbound payment eventsâ

- **Customer data**: `customers`
  - **Key**: `sender_id` and `receiver_id`
- **Account data**: `accounts`
  - **Key**: `sender_account_iban` and `receiver_account_iban`
- **Device data**: `devices`
  - **Key**: `device_id`

Outbound transfer eventsâ

- **Customer data**: `customers`.
  - **Key**: `sender_id` and `receiver_id`
- **Account data**: `accounts`.
  - **Key**: `sender_account_iban` and `receiver_account_iban`.
- **Device data**: `devices`
  - **Key**: `device_id`